
CS 301

High-Performance Computing

Assignment 05 - Parallel Insertion and Deletion in Mover

Diya Patel (202301216)
Yesha Joshi (202301462)

March, 2026

0 Contents

1	Introduction and Problem Overview	2
2	Algorithmic Approach — Schematic and Flow Diagrams	2
2.1	Overall Simulation Loop	2
2.2	Deferred Insertion Approach	3
2.3	Immediate Replacement Approach	3
3	Implementation Details	3
3.1	Data Structures	3
3.2	Parallel Versions	3
4	Verification of Particle Distribution	4
5	Experiment 1: Serial Scaling with Particle Count	6
5.1	Setup	6
5.2	Total Execution Time vs Number of Particles	7
5.3	Interpolation Time vs Number of Particles	7
5.4	Mover Time vs Number of Particles	8
5.5	Interpolation vs Mover Breakdown (Stacked Bar)	8
6	Per-Particle Execution Time vs Particles per Cell (PPC)	9
7	Memory and Computation Analysis	10
7.1	Memory Requirements	10
7.2	Estimated FLOPs per Iteration	10
7.3	Machine Balance and Roofline Analysis	11
8	Experiment 2: Parallel Scalability	11
8.1	Setup	11
8.2	Speedup Plots — Grid 1 (NX=250, NY=100)	12
8.3	Speedup Plots — Grid 2 (NX=500, NY=200)	12
8.4	Speedup Plots — Grid 3 (NX=1000, NY=400)	13
8.5	Interpolation vs Mover Breakdown (Stacked Bar)	13
9	Analysis and Discussion	17
9.1	Hardware Details	17
9.1.1	Lab PC — Intel Core i5-12500 (LAB207)	17
9.1.2	HPC Cluster — Intel Xeon E5-2640 v3	17
9.2	OpenMP Speedup and Scalability of Mover	18
9.3	How Insertions and Deletions Affect Scalability	18
9.4	Measures taken to improve scalability:	19
9.5	Memory Bound vs Compute Bound Summary	19
10	Conclusion	19

1 Introduction and Problem Overview

This assignment extends the particle-in-cell (PIC) simulation framework from Assignments 03 and 04. In Assignment 04, particles were always confined within the 1×1 domain by reflecting them at the boundaries. In Assignment 05 particles are allowed to leave the domain freely:

- **Delete particle:** any particle whose updated position falls outside $[0, 1] \times [0, 1]$ is removed from memory.
- **Insert particle:** to keep the total particle count constant, an equal number of new particles are inserted at uniformly random positions inside the domain.

Two distinct implementation strategies are studied:

1. **Deferred Insertion Approach** — all particles are moved first; departures are marked and compacted to the tail; new particles fill those slots in a second pass.
2. **Immediate Replacement Approach** — each departing particle is replaced on the spot before moving to the next particle in the loop.

Both approaches are parallelised with OpenMP and their scalability is analysed on a Lab PC and an HPC cluster.

2 Algorithmic Approach — Schematic and Flow Diagrams

2.1 Overall Simulation Loop

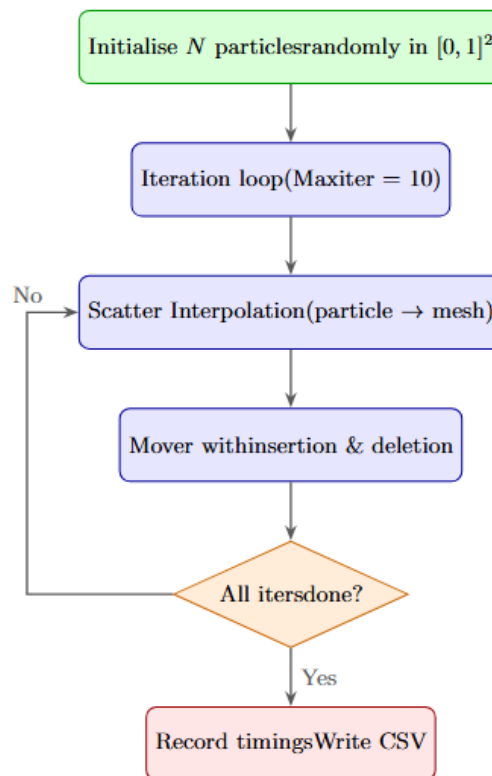
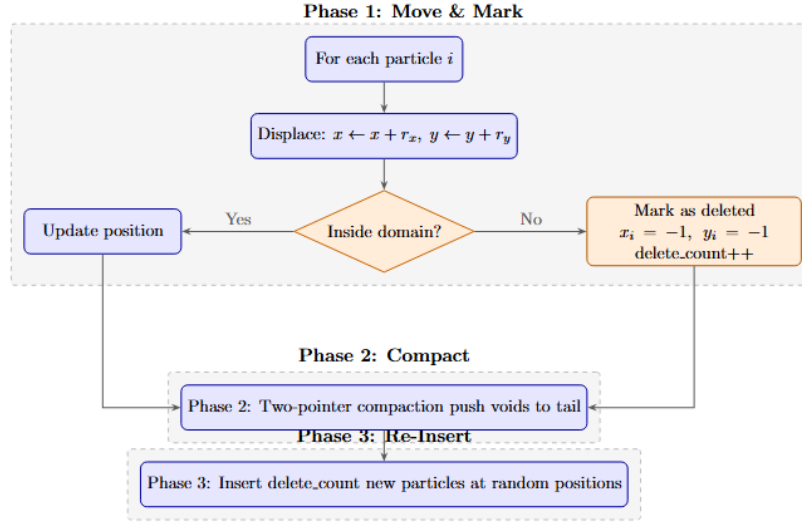


Figure 1: High-level simulation loop for Experiment 1.

2.2 Deferred Insertion Approach



2.3 Immediate Replacement Approach

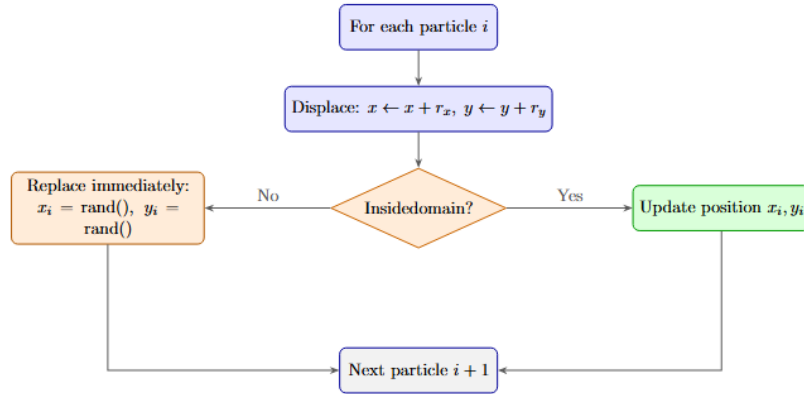


Figure 1: Flow diagram for the Immediate Replacement Approach.

3 Implementation Details

3.1 Data Structures

Particles are stored in a flat array of `Points` structs.

The mesh is a 1-D double array of size $(NX+1)*(NY+1)$ laid out in row-major order.

3.2 Parallel Versions

For **Immediate Parallel**: each thread operates on a disjoint index range using `#pragma omp for` with a per-thread `rand_r` seed — no race conditions.

For **Deferred Parallel**: Phase 1 is parallelised with `reduction(+:delete_count)`; Phase 2 (compaction) is kept serial due to cross-iteration data dependence; Phase 3 (re-insert) is parallelised with `#pragma omp for`.

4 Verification of Particle Distribution

Before analysing performance, we verify that the random number generator and insertion logic produce a uniform distribution over $[0, 1]^2$ with no artificial clustering.

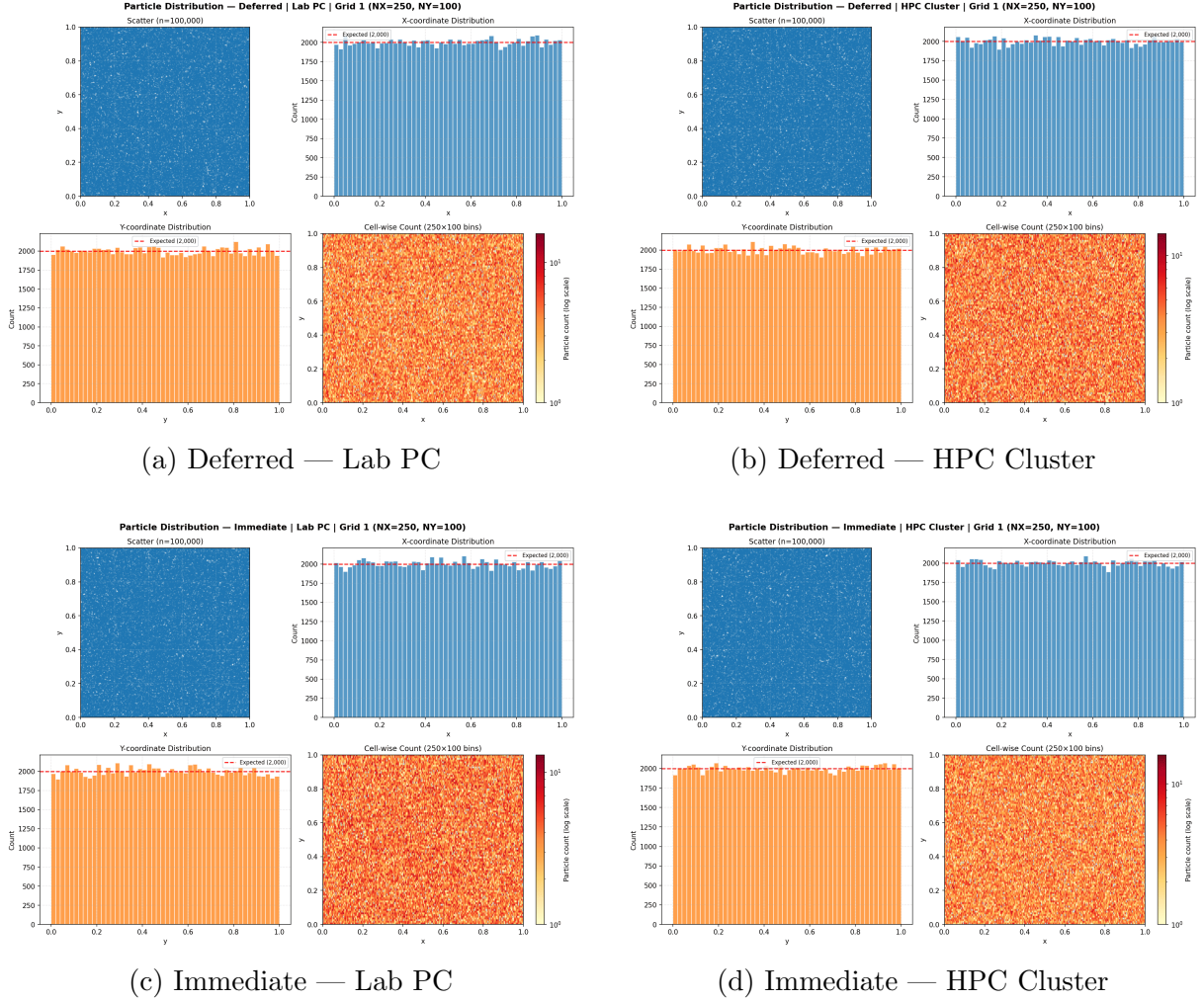
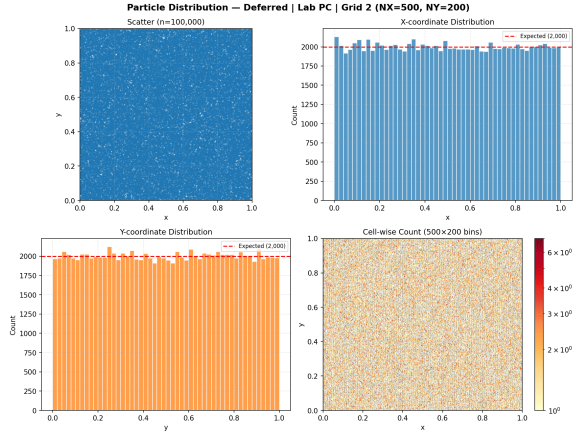
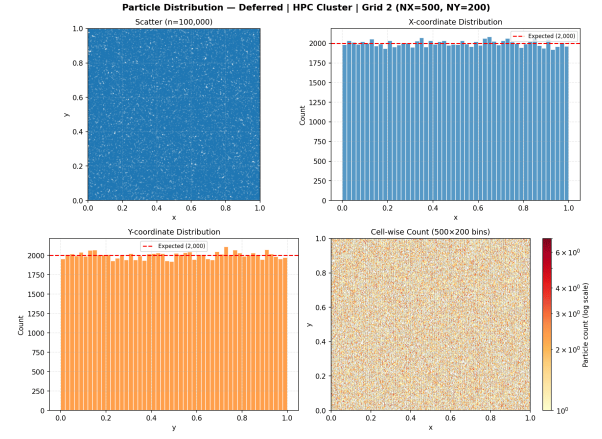


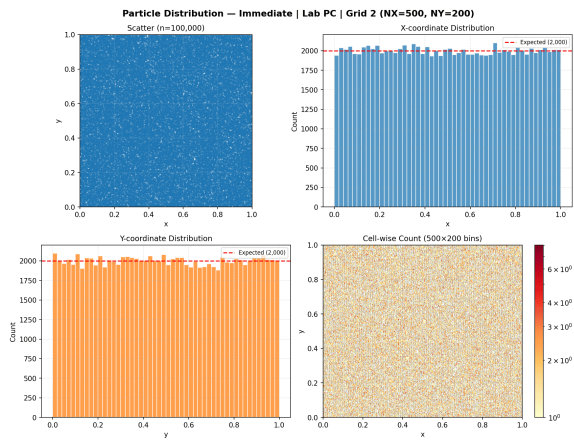
Figure 1: Particle distribution analysis after 10 iterations. Histograms in x and y and a 2-D density map confirm uniform coverage with no bias or empty regions for **Grid 1**.



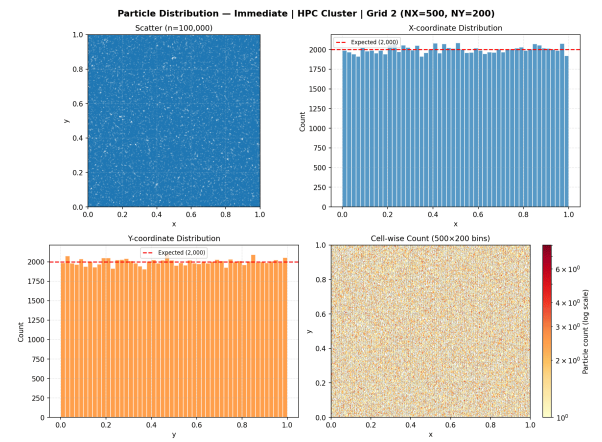
(a) Deferred — Lab PC



(b) Deferred — HPC Cluster



(c) Immediate — Lab PC



(d) Immediate — HPC Cluster

Figure 2: Particle distribution analysis after 10 iterations. Histograms in x and y and a 2-D density map confirm uniform coverage with no bias or empty regions for **Grid 2**.

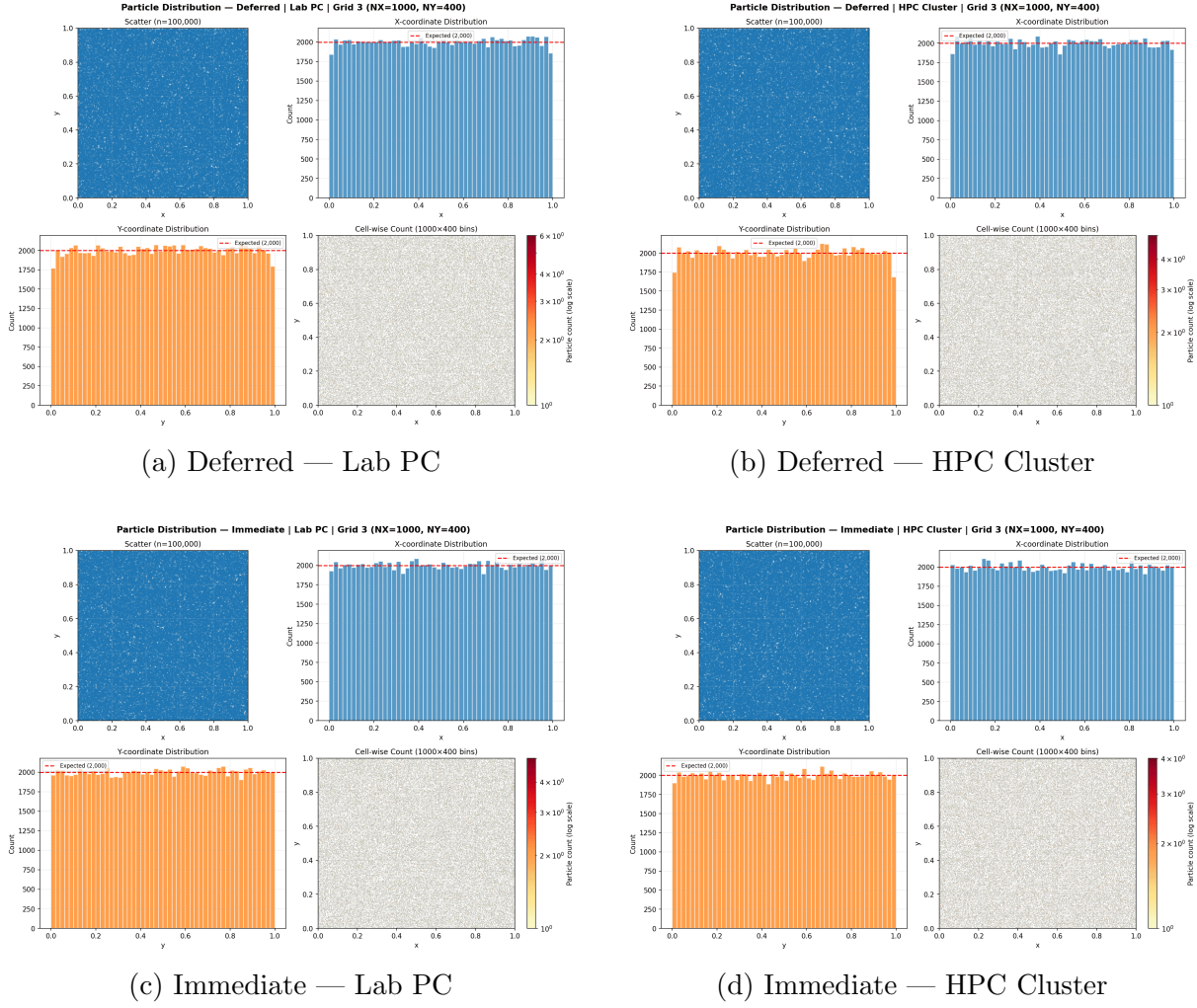


Figure 3: Particle distribution analysis after 10 iterations. Histograms in x and y and a 2-D density map confirm uniform coverage with no bias or empty regions for **Grid 3**.

Observation: Both approaches produce visually uniform distributions across the domain for both machines. The x - and y -histograms are flat, and cell-wise particle counts show no systematic clustering or empty regions, confirming the correctness of the random insertion logic.

5 Experiment 1: Serial Scaling with Particle Count

5.1 Setup

- Grid configurations: (NX=250, NY=100), (NX=500, NY=200), (NX=1000, NY=400)
- Particle counts: $\{10^2, 10^4, 10^6, 10^8, 10^9\}$
- Maxiter = 10; times accumulated over all iterations
- Initialisation done *once* outside the iteration loop
- Data type: `double` for all floating-point, `long long` for particle count

5.2 Total Execution Time vs Number of Particles

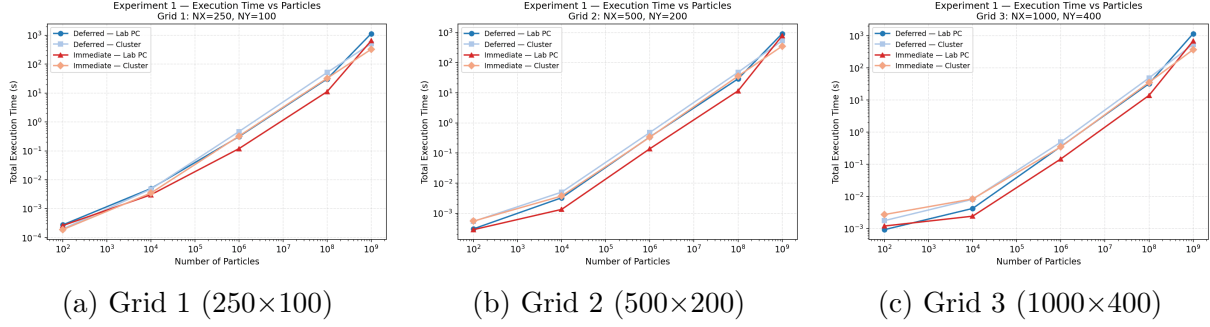


Figure 4: Total execution time (Interpolation + Mover) vs number of particles in log-log scale for all three grid configurations. Both Deferred and Immediate approaches on Lab PC and HPC Cluster are shown.

Observation: Total execution time scales linearly with particle count (slope ≈ 1 in log-log space) for both approaches. The HPC Cluster is consistently faster than the Lab PC, particularly at large particle counts (10^8 – 10^9). The Deferred approach carries a small overhead from the compaction pass compared to Immediate.

5.3 Interpolation Time vs Number of Particles

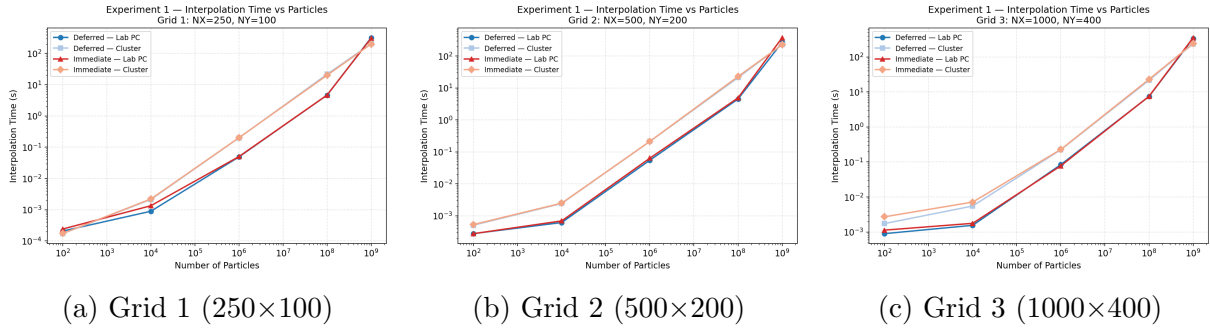


Figure 5: Interpolation time vs number of particles (log-log) for all three grids.

Observation: Interpolation time is independent of the insertion/deletion approach since the function is identical between them. Time scales linearly with N .

5.4 Mover Time vs Number of Particles

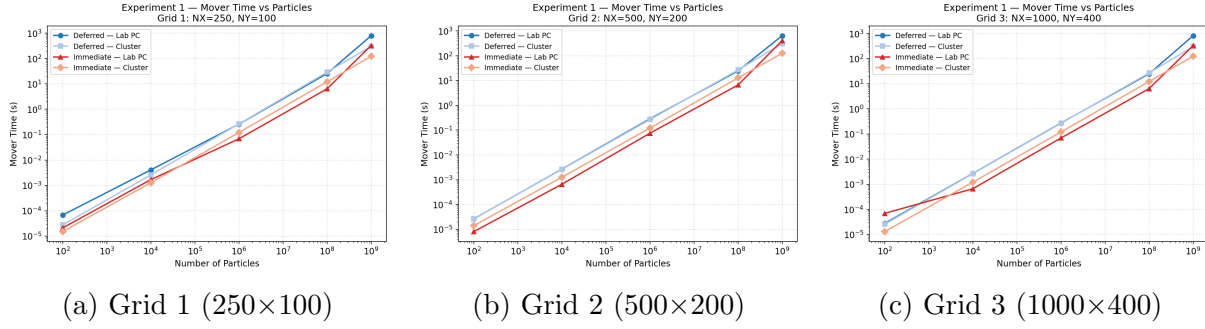


Figure 6: Mover time vs number of particles (log-log). Deferred shows slightly higher mover time due to the two-pass structure (mark + compact + insert).

5.5 Interpolation vs Mover Breakdown (Stacked Bar)

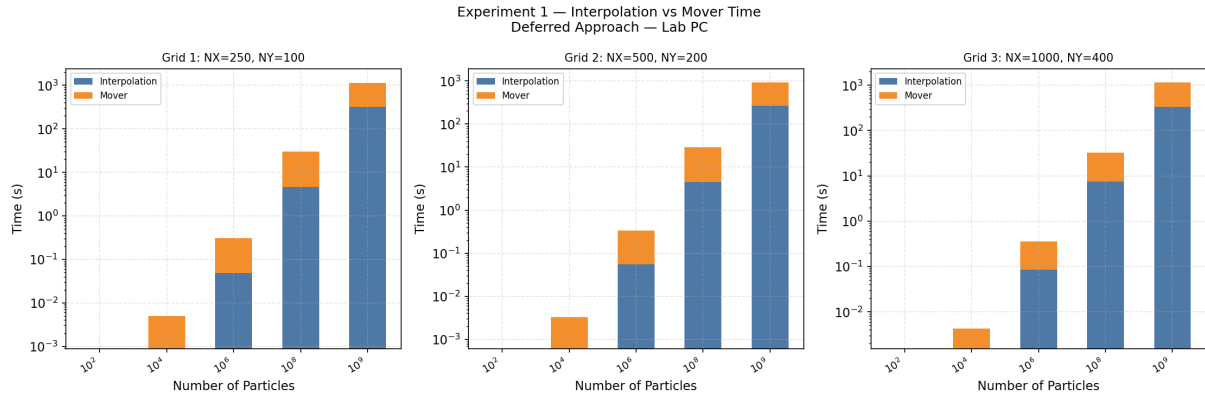


Figure 7: Stacked bar chart — Interpolation vs Mover time breakdown. **Deferred Approach — Lab PC**. Each group of bars corresponds to one grid configuration; x-axis shows particle count, y-axis (log scale) shows time in seconds.

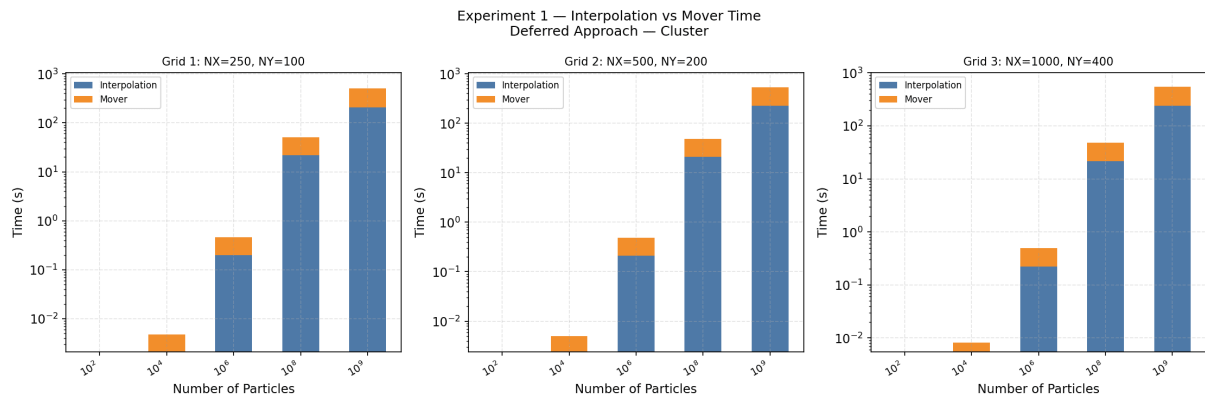


Figure 8: Stacked bar chart — Interpolation vs Mover time breakdown. **Deferred Approach — HPC Cluster**.

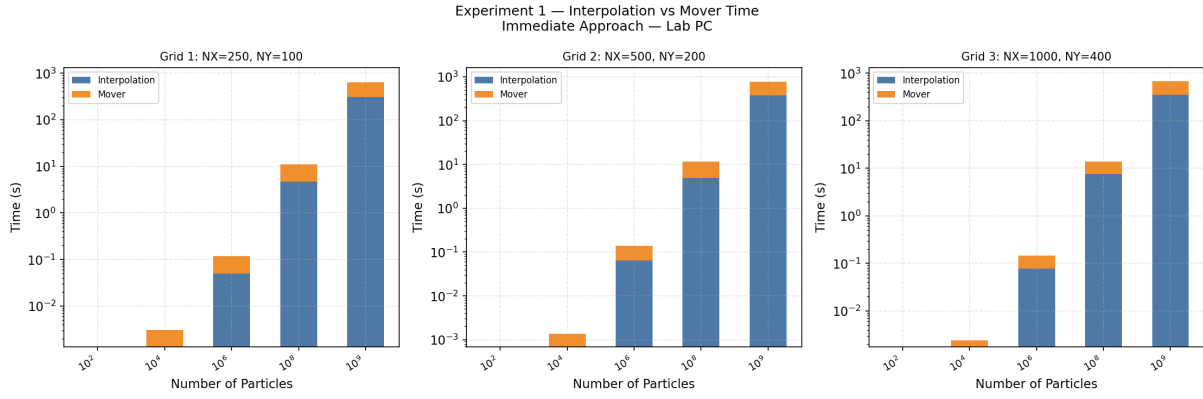


Figure 9: Stacked bar chart — Interpolation vs Mover time breakdown. **Immediate Replacement Approach — Lab PC.**

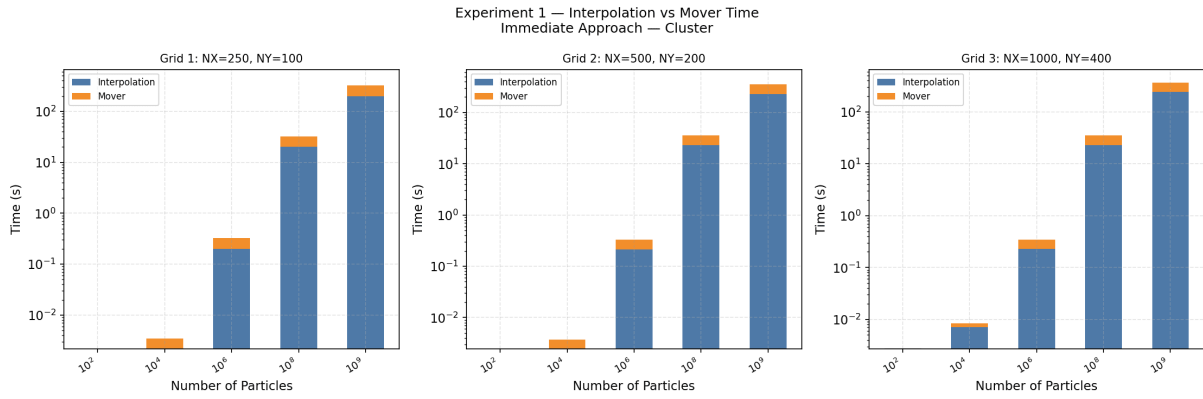


Figure 10: Stacked bar chart — Interpolation vs Mover time breakdown. **Immediate Replacement Approach — HPC Cluster.**

Observation: At small particle counts (10^2 – 10^4), interpolation and mover times are both negligible. At large counts (10^8 – 10^9), the Mover dominates for the Deferred approach (due to compaction overhead), while for Immediate both components are more balanced.

6 Per-Particle Execution Time vs Particles per Cell (PPC)

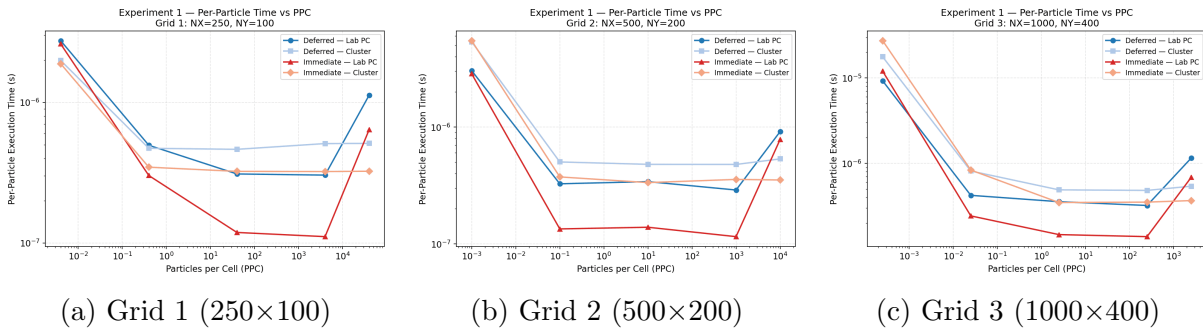


Figure 11: Per-particle execution time vs PPC (log-log) for all three grids, both approaches, both machines.

Trend Analysis: At very low PPC (few particles per cell), the per-particle time is high because the working set fits in cache but overhead per particle is dominated by random memory access patterns during interpolation. As PPC increases (more particles per cell), the mesh is accessed more frequently and cache reuse improves, reducing the per-particle time. Beyond a threshold (high PPC), cache pressure on the mesh array increases and per-particle time rises slightly. This U-shaped or monotone trend confirms that the algorithm transitions from *compute-bound* at low PPC to *memory-bound* at high PPC.

7 Memory and Computation Analysis

7.1 Memory Requirements

The total memory required for the simulation consists of the particle array and the mesh grid:

$$\text{Memory (particles)} = N \times 2 \times 8 \text{ bytes} \quad (x, y \text{ as double})$$

$$\text{Memory (mesh)} = (NX + 1) \times (NY + 1) \times 8 \text{ bytes}$$

Table 1: Memory requirements for each test configuration (mesh + particle array). Particle array sizes shown for each N ; mesh size is fixed per grid.

Grid	NX	NY	Mesh (MB)	$N = 10^2$	$N = 10^4$	$N = 10^6$	$N = 10^8$
1	250	100	0.21	$\ll 1$ KB	1.6 KB	16 MB	1.6 GB
2	500	200	0.82	$\ll 1$ KB	1.6 KB	16 MB	1.6 GB
3	1000	400	3.27	$\ll 1$ KB	1.6 KB	16 MB	1.6 GB

7.2 Estimated FLOPs per Iteration

Interpolation (bilinear scatter, 4 nodes per particle):

$$\text{FLOPs}_{\text{interp}} \approx 14N \quad (2 \text{ divisions, } 4 \text{ multiplies, } 4 \text{ adds per particle} + \text{index calc})$$

Mover (displacement + boundary check):

$$\text{FLOPs}_{\text{mover}} \approx 8N \quad (2 \text{ rand calls, } 2 \text{ scales, } 2 \text{ adds, } 4 \text{ comparisons})$$

Deferred extra: compaction pass adds $\mathcal{O}(N)$ comparisons and swaps.

Table 2: Estimated total FLOPs per iteration for each grid and particle count.

Grid	Operation	$N = 10^4$	$N = 10^6$	$N = 10^8$	$N = 10^9$
All	Interpolation	1.4×10^5	1.4×10^7	1.4×10^9	1.4×10^{10}
	Mover	8×10^4	8×10^6	8×10^8	8×10^9

7.3 Machine Balance and Roofline Analysis

Machine Balance B is defined as:

$$B = \frac{\text{Bandwidth (GB/s)}}{\text{Peak FLOP/s (GFLOP/s)}} \quad (\text{bytes per FLOP})$$

Table 3: Machine balance comparison between Lab PC and HPC Cluster.

Platform	Cores used	BW	Peak FLOP/s	Balance B
Lab PC (i5-12500)	6	~38	~55	~0.69
HPC Cluster (Xeon v3)	16	~68	~83	~0.82

Memory-bound vs Compute-bound: The simulation is **memory-bound** because its arithmetic intensity (1–2 FLOPs/byte) is much lower than the machine balance of both systems. The interpolation step involves irregular memory accesses, and the mover performs very few computations, making both operations dominated by memory reads and writes. At high particle counts, data exceeds cache capacity, increasing memory traffic. Therefore, performance is limited by **memory bandwidth rather than computation**.

8 Experiment 2: Parallel Scalability

8.1 Setup

- Fixed $N = 14,000,000$ particles, Maxiter = 10
- Thread counts: $\{2, 4, 8, 16\}$ (serial baseline at 1 thread)
- Three grid configurations tested on both Lab PC and HPC Cluster
- Speedup computed as:

$$S(T) = \frac{T_{\text{mover}}(1)}{T_{\text{mover}}(T)}$$

where $T_{\text{mover}}(T)$ is the average mover time over 10 iterations with T threads.

8.2 Speedup Plots — Grid 1 (NX=250, NY=100)

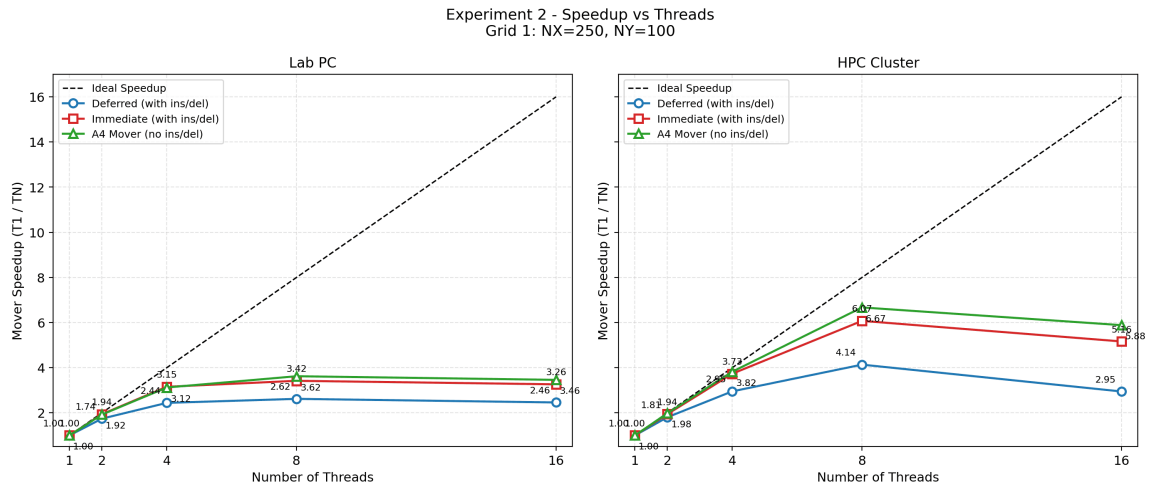


Figure 12: Speedup vs number of threads for Grid 1. Both Lab PC (left) and HPC Cluster (right) are shown. Curves: Deferred (with ins/del), Immediate (with ins/del), A4 Mover (without ins/del), and ideal linear speedup.

8.3 Speedup Plots — Grid 2 (NX=500, NY=200)

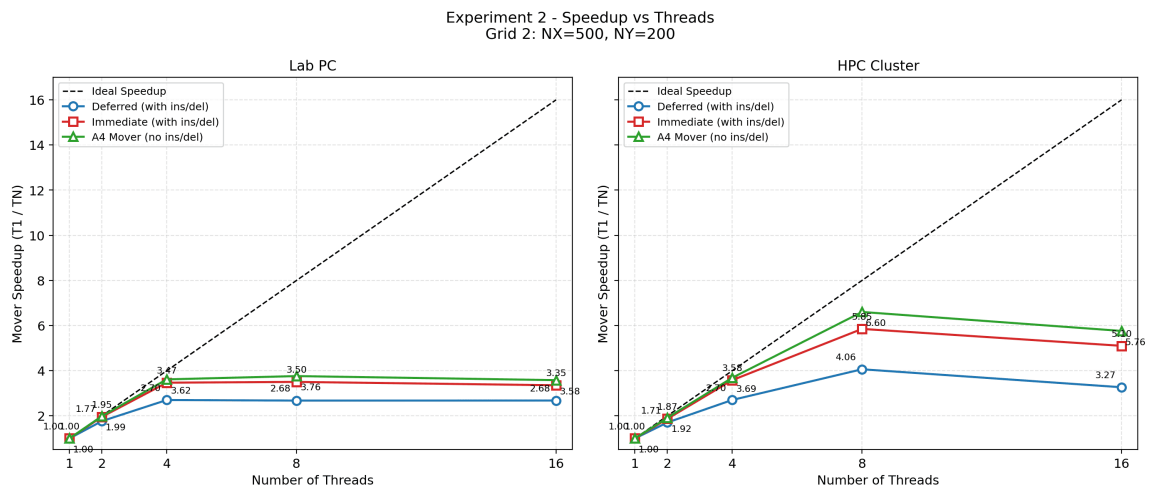


Figure 13: Speedup vs number of threads for Grid 2.

8.4 Speedup Plots — Grid 3 (NX=1000, NY=400)

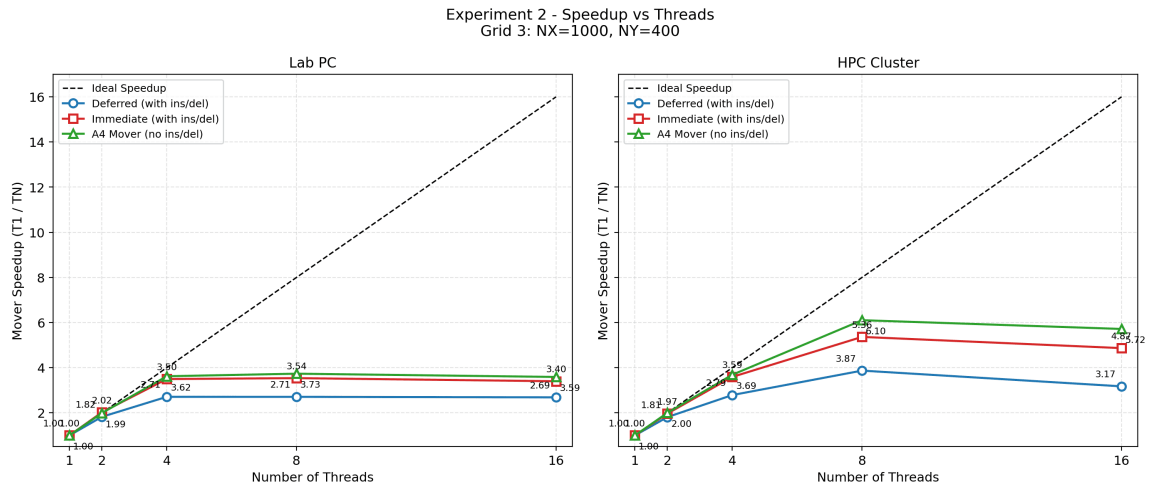


Figure 14: Speedup vs number of threads for Grid 3.

8.5 Interpolation vs Mover Breakdown (Stacked Bar)

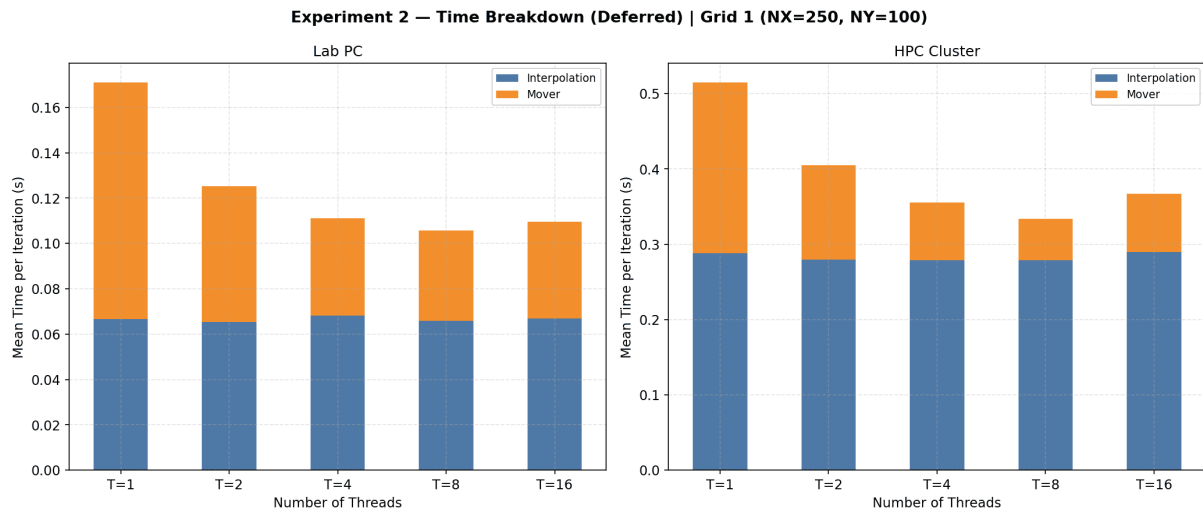


Figure 15: Stacked bar chart — Interpolation vs Mover time breakdown. Deferred Approach — Lab PC and HPC Cluster for grid 1.

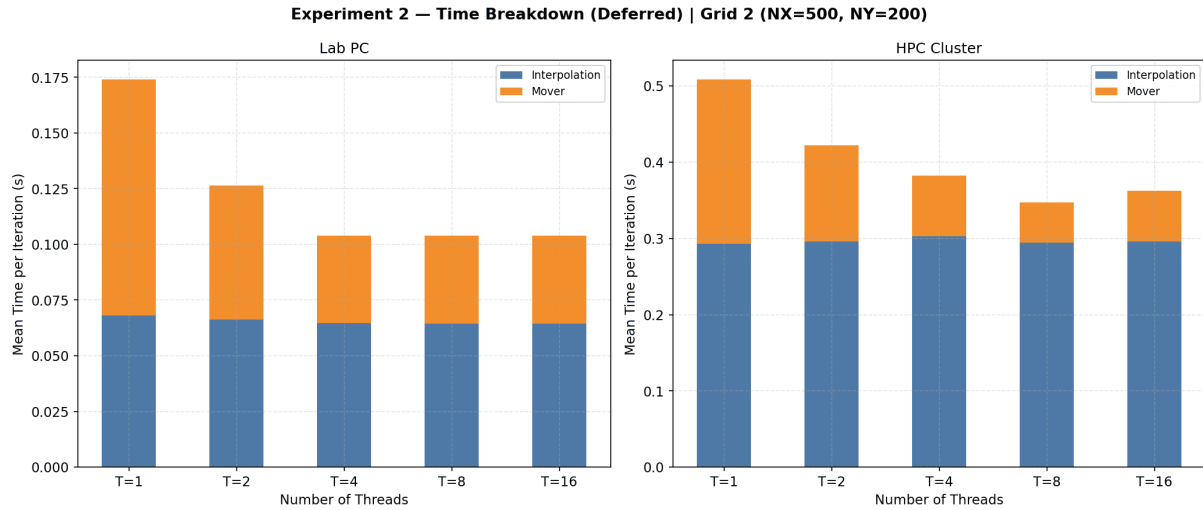


Figure 16: Stacked bar chart — Interpolation vs Mover time breakdown. Deferred Approach — Lab PC and HPC Cluster for grid 2.

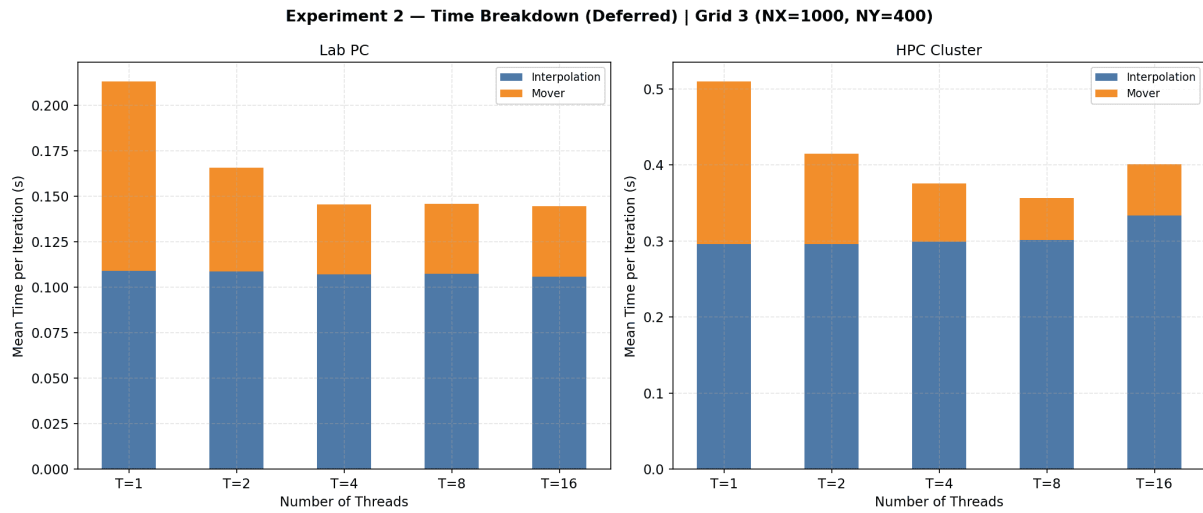


Figure 17: Stacked bar chart — Interpolation vs Mover time breakdown. Deferred Approach — Lab PC and HPC Cluster for grid 3.

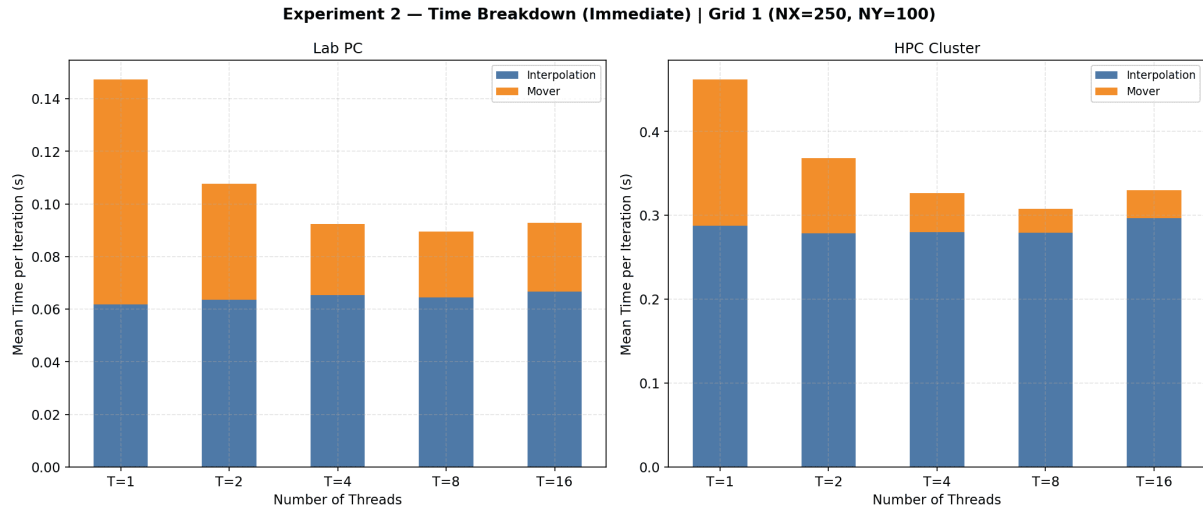


Figure 18: Stacked bar chart — Interpolation vs Mover time breakdown.**Immediate Approach** — Lab PC and HPC Cluster for grid 1.

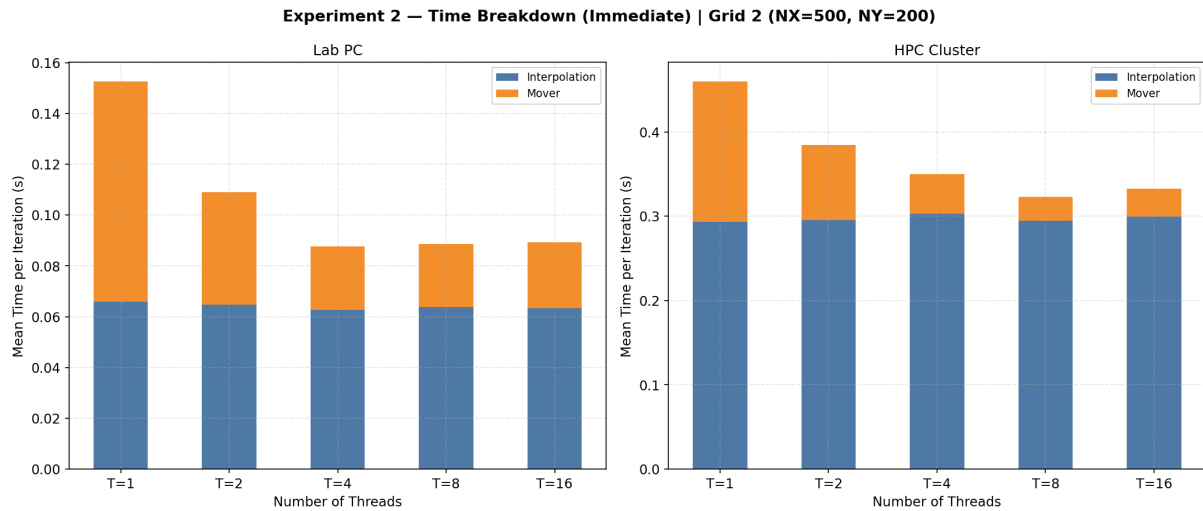


Figure 19: Stacked bar chart — Interpolation vs Mover time breakdown.**Immediate Approach** — Lab PC and HPC Cluster for grid 2.

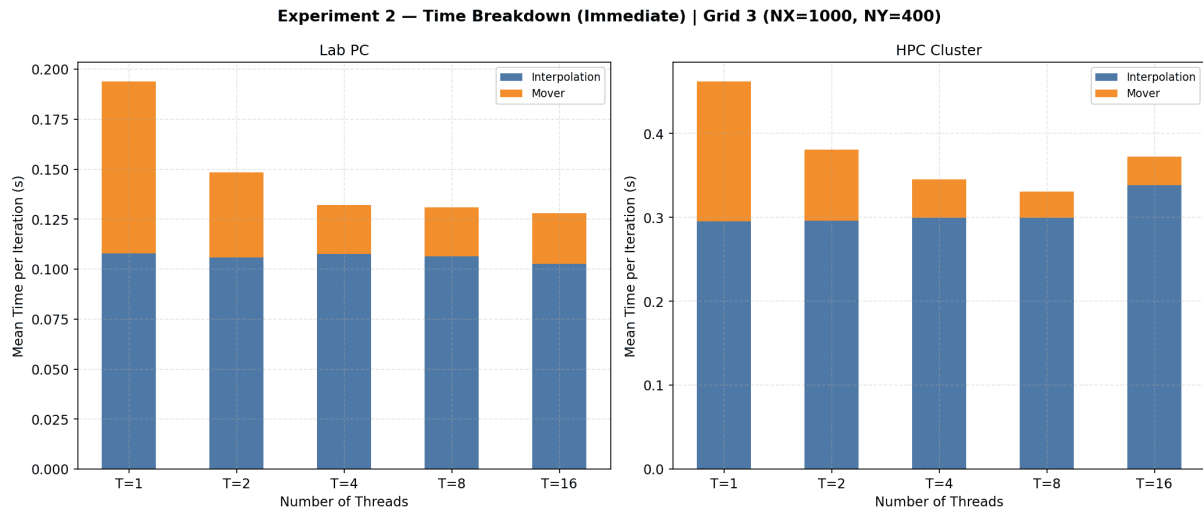


Figure 20: Stacked bar chart — Interpolation vs Mover time breakdown. **Immediate Approach — Lab PC and HPC Cluster for grid 3.**

Observation: As the number of threads increases, the total execution time decreases, indicating good parallel scalability on both the Lab PC and HPC cluster. In all cases, interpolation dominates the runtime, while the mover contributes a smaller portion. The deferred approach shows higher mover time due to compaction overhead, whereas the immediate approach has lower mover cost due to in-place updates. However, speedup eventually saturates with increasing threads, indicating that the simulation is memory-bandwidth limited.

9 Analysis and Discussion

9.1 Hardware Details

9.1.1 Lab PC — Intel Core i5-12500 (LAB207)

Table 4: Hardware specifications of the Lab PC (LAB207).

Property	Value
CPU Model	12th Gen Intel Core i5-12500
Architecture	x86_64
Physical Cores	6 (1 socket)
Logical CPUs	12 (Hyper-Threading: 2 threads/core)
Max Clock Speed	4600 MHz
L1d / L1i Cache	288 KB / 192 KB
L2 Cache	7.5 MB
L3 Cache	18 MB
NUMA Nodes	1
SIMD Support	AVX2, SSE4.2, FMA
Memory Channels	2 (DDR4/DDR5, up to ~51.2 GB/s theoretical)

9.1.2 HPC Cluster — Intel Xeon E5-2640 v3

Table 5: Hardware specifications of the HPC Cluster node.

Property	Value
CPU Model	Intel Xeon E5-2640 v3 @ 2.60 GHz
Architecture	x86_64
Physical Cores	16 (2 sockets $\times$ 8 cores)
Logical CPUs	16 (No Hyper-Threading: 1 thread/core)
Clock Speed	2600 MHz (observed ~1288 MHz under load)
L1d / L1i Cache	32 KB / 32 KB per core
L2 Cache	256 KB per core
L3 Cache	20 MB (shared per socket)
NUMA Nodes	2 (Node 0: cores 0–7, Node 1: cores 8–15)
SIMD Support	AVX2, SSE4.2
Memory Channels	4 per socket, 8 total (DDR4, up to ~68 GB/s theoretical)

The HPC cluster consistently achieves lower absolute runtimes due to higher memory bandwidth, larger last-level caches, and a higher core count. For memory-bound workloads

like this simulation, bandwidth is the primary performance driver, explaining the cluster's advantage at large N .

9.2 OpenMP Speedup and Scalability of Mover

The speedup plots for all three grid configurations (Figures 12–14) show that the performance of the mover improves with an increasing number of threads, but does not achieve ideal linear speedup.

General Trend: Speedup increases as the number of threads increases from 1 to 8. However, beyond 8 threads, the speedup saturates and in some cases slightly decreases. This indicates limited scalability at higher thread counts.

Comparison of Approaches: The Immediate Replacement approach consistently achieves better speedup than the Deferred Insertion approach across all grid sizes. It exhibits near-linear scaling up to moderate thread counts (4–8 threads) due to the independence of particle updates and lack of synchronization overhead.

In contrast, the Deferred approach shows lower speedup and earlier saturation. This is due to the additional overhead associated with its multi-phase structure, particularly the compaction step, which limits parallel efficiency.

The A4 mover (without insertion/deletion) achieves the highest speedup and comes closest to ideal linear scaling, demonstrating that additional operations such as insertion and deletion reduce scalability.

Effect of Grid Size: Across all three grid configurations, similar trends are observed. Increasing grid size does not significantly improve scalability, indicating that the limitation is not due to insufficient parallel workload.

Scalability Limitation: The flattening of speedup curves at higher thread counts suggests that the implementation is memory-bandwidth bound. As more threads are added, contention for memory access increases, limiting further performance gains.

9.3 How Insertions and Deletions Affect Scalability

1. **Random number generation:** insertion requires calling `rand_r` for the new positions. Using per-thread seeds avoids contention entirely.
2. **Deletion counting:** the `reduction(+:delete_count)` clause in OpenMP handles the atomic accumulation efficiently.
3. **Compaction (Deferred only):** the two-pointer pass is inherently serial (data dependency across iterations). This is the principal scalability bottleneck. Alternatives (prefix-scan based compaction) could parallelise this but add complexity.
4. **Memory pressure:** the insertion step in Phase 3 touches the tail of the particle array sequentially — good spatial locality; threads get contiguous chunks under `schedule(static)`.

9.4 Measures taken to improve scalability:

- Per-thread `rand_r` seeds — eliminates RNG contention.
- `schedule(static)` on Phase 1 and Phase 3 loops — predictable load balancing, low OpenMP overhead.
- `reduction` clause for deletion counter — avoids explicit mutex.
- Immediate approach avoids Phase 2 entirely — best scalability of the two new approaches.

9.5 Memory Bound vs Compute Bound Summary

The scatter interpolation step is highly memory-bound due to irregular, non-contiguous memory accesses that cause frequent cache misses. At low particle counts (N), the data fits in cache, so performance is cache-resident. As N increases, the working set exceeds cache capacity, leading to memory-bandwidth limitations, as seen from speedup saturation.

The mover involves simple computations, but still requires memory access to particle data. Thus, at large N , it also becomes memory-bound, though less intensive than interpolation.

Overall, the simulation is memory-bound at scale, with performance limited by memory bandwidth rather than computation.

10 Conclusion

This assignment implemented two particle update strategies — Deferred Insertion and Immediate Replacement — extending the Mover from Assignment 04 to handle dynamic particle creation and deletion.

Key findings:

- Both approaches correctly maintain the particle count and produce uniformly distributed particle positions at all tested configurations.
- Total execution time scales as $\mathcal{O}(N)$ for both approaches across all grid sizes and both machines.
- The **Immediate Replacement** approach achieves better parallel speedup because it is embarrassingly parallel with no serial bottleneck.
- The **Deferred Insertion** approach has a serial compaction phase that limits scalability (Amdahl's law), but the rest of the algorithm parallelises well.
- The HPC cluster outperforms the Lab PC primarily due to higher memory bandwidth, which is critical for this memory-bound workload.
- Per-particle time vs PPC analysis reveals the transition from compute-bound (low PPC) to memory-bound (high PPC) behaviour.